

LAPORAN PRAKTIKUM
Class, Object, dan Encapsulation Part 2

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah
Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA : Maulida Nur Afifah
NIM : 251511017
KELAS : 2-A
PROGRAM STUDI : D3 Teknik Informatika
JURUSAN : Teknik Komputer dan Informatika

POLITEKNIK NEGERI BANDUNG

2026

Kasus:

Sebuah kantin kampus membutuhkan program, sederhana untuk mencatat pesanan. Seorang mahasiswa memilih satu menu. Setiap menu mempunyai kode nama. Harga, dan kondisi tersedia/habis. Sebuah pesanan menyimpan siapa pemesannya, menu yang dipilih, dan jumlah yang dipesan. Total dihitung dari harga menu x jumlah. Setiap pesanan memiliki nomor urut yang dibuat otomatis.

Batasan kasus:

- satu pesanan hanya satu jenis menu
- tidak ada database, login, pembayaran, diskon, stok numerik, gui, atau banyak tenant
- status menu hanya tersedia/habis

Tahap 1 – Bangun Class Mahasiswa

Mahasiswa.java

```
public class Mahasiswa {  
    private String nim;  
    private String nama;  
  
    public Mahasiswa(String nim, String nama) {  
        this.nim = nim;  
        this.nama = nama;  
    }  
  
    public String getNim() {  
        return nim;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) {  
  
        Mahasiswa m1 = new Mahasiswa("241001", "Asep");  
        Mahasiswa m2 = new Mahasiswa("241002", "Siti");  
  
        System.out.println(m1.getNim() + " - " + m1.getNama());  
        System.out.println(m2.getNim() + " - " + m2.getNama());  
    }  
}
```

Output:

```
D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>java id.ac.polban.pbo.kantin.app.Main  
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8  
241001 - Asep  
241002 - Siti
```

Pertanyaan	Jawaban
Apakah m1 dan 2 menggunakan class yang sama?	Ya
Mengapa statenya bisa berbeda?	Karena setiap new membuat object baru
Apa yang terjadi jika System.out.println(m1.nama)?	Error karena nama private tidak bisa diakses langsung dari luar class (perlu getName)

Tahap 2 – Bangun MenuItem dan Uji Encapsulation

MenuItem.java

```

public class MenuItem {

    private String kode;
    private String nama;
    private int harga;
    private boolean tersedia;

    public MenuItem(String kode, String nama, int harga) {
        this.kode = kode;
        this.nama = nama;
        this.harga = harga;
        this.tersedia = true; // otomatis true saat dibuat
    }

    public String getKode() {
        return kode;
    }

    public String getName() {
        return nama;
    }

    public int getHarga() {
        return harga;
    }

    public boolean isTersedia() {
        return tersedia;
    }

    public void tandaiHabis() {
        tersedia = false;
    }

}

```

Main.java

```

public class Main {
    public static void main(String[] args) {

        // Tahap 1
        Mahasiswa m1 = new Mahasiswa("241001", "Asep");
        Mahasiswa m2 = new Mahasiswa("241002", "Siti");

        System.out.println("=== Data Mahasiswa ===");
        System.out.println(m1.getNim() + " - " + m1.getName());
        System.out.println(m2.getNim() + " - " + m2.getName());
        System.out.println();

        // Tahap 2
        MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
        MenuItem kopi = new MenuItem("M02", "Kopi Susu", 12000);
    }
}

```

```

        kopi.tandaiHabis();

        System.out.println("=== Status Menu ===");
        System.out.println(nasi.getNama() + " tersedia: " + nasi.isTersedia());
        System.out.println(kopi.getNama() + " tersedia: " + kopi.isTersedia());

        // Eksperimen wajib:
        // kopi.tersedia = false;
    }
}

```

Output:

```

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>java id.ac.polban.pbo.kantin.app.Main
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Data Mahasiswa ===
241001 - Asep
241002 - Siti

=== Status Menu ===
Nasi Goreng tersedia: true
Kopi Susu tersedia: false

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>

```

Eksperimen wajib:

```

44
45 // Eksperimen wajib
46 kopi.tersedia = false;
47
48 }

```

Problems Output Debug Console Terminal Ports Spell Checker 81 Comments

```

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>javac id/ac/polban/pbo/kantin/app/Main.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
id\ac\polban\pbo\kant\app\Main.java:46: error: tersedia has private access in MenuItem
    kopi.tersedia = false;
        ^
1 error

```

Field private tidak boleh diakses langsung dari luar class

Tahap 3 – Bangun Pesanan dan Relationship

Pesanan.java

```

public class Pesanan {

    private static int nextNumber = 1;

    private int nomor;
    private Mahasiswa pemesan; // reference ke object Mahasiswa
    private MenuItem menu; // reference ke object MenuItem
    private int jumlah;

    public Pesanan(Mahasiswa pemesan, MenuItem menu, int jumlah) {
        this.nomor = nextNumber++;
        this.pemesan = pemesan;
        this.menu = menu;
    }
}

```

```

        this.jumlah = jumlah;
    }

    public boolean dapatDiproses() {
        // pesan ke MenuItem – Pesanan tidak perlu tahu cara MenuItem menyimpan data
        return jumlah > 0 && menu.isTersedia();
    }

    public int hitungTotal() {
        return menu.getHarga() * jumlah;
    }

    public void ringkasan() {
        System.out.println("=== Pesanan #" + nomor + " ===");
        System.out.println("Pemesan : " + pemesan.getNama() + " (" + pemesan.getNim() + ")");
        System.out.println("Menu      : " + menu.getNama());
        System.out.println("Jumlah   : " + jumlah);
        System.out.println("Total    : Rp " + hitungTotal());
        System.out.println("Status   : " + (dapatDiproses() ? "Dapat diproses" : "Ditolak"));
    }
}

```

Tahap 4 – Buat dan Jalankan Object yang Berkolaborasi

Main.java

```

public class Main {
    public static void main(String[] args) {

        // Mahasiswa
        Mahasiswa m1 = new Mahasiswa("241001", "Asep");
        Mahasiswa m2 = new Mahasiswa("241002", "Siti");

        System.out.println("=== Data Mahasiswa ===");
        System.out.println(m1.getNim() + " - " + m1.getNama());
        System.out.println(m2.getNim() + " - " + m2.getNama());
        System.out.println();

        // MenuItem
        MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
        MenuItem kopi = new MenuItem("M02", "Kopi Susu", 12000);

        kopi.tandaiHabis();

        System.out.println("=== Status Menu ===");
        System.out.println(nasi.getNama() + " tersedia: " + nasi.isTersedia());
        System.out.println(kopi.getNama() + " tersedia: " + kopi.isTersedia());
        System.out.println();

        // Pesanan [BARU]
        Pesanan p1 = new Pesanan(m1, nasi, 2);
        Pesanan p2 = new Pesanan(m2, kopi, 1); // kopi sudah habis

        System.out.println("=== Hasil Pesanan ===");
        System.out.println("P1 dapat diproses : " + p1.dapatDiproses());
        System.out.println("Total P1      : Rp " + p1.hitungTotal());
        System.out.println("P2 dapat diproses : " + p2.dapatDiproses());
        System.out.println();

        p1.ringkasan();
        System.out.println();
        p2.ringkasan();
    }
}

```

Output:

```
D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>java id.ac.polban.pbo.kantin.app.Main
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Data Mahasiswa ===
241001 - Asep
241002 - Siti

=== Status Menu ===
Nasi Goreng tersedia: true
Kopi Susu tersedia: false

=== Hasil Pesanan ===
P1 dapat diproses : true
Total P1          : Rp 36000
P2 dapat diproses : false

=== Pesanan #1 ===
Pemesan : Asep (241001)
Menu    : Nasi Goreng
Jumlah  : 2
Total   : Rp 36000
Status  : Dapat diproses

=== Pesanan #2 ===
Pemesan : Siti (241002)
Menu    : Kopi Susu
Jumlah  : 1
```

Pertanyaan	Jawaban
Mengapa p1 dan p2 bisa punya pemesan berbeda?	Karena masing-masing Pesanan menyimpan reference ke object Mahasiswa yang berbeda
Mengapa p2 ditolak setelah kopi ditandai habis?	dapatDiproses() memanggil menu.isTersedia() karena kopi sudah tandaiHabis(), hasilnya false
Object mana yang mengetahui harga?	MenuItem. Pesanan hanya memanggil menu.getHarga(), tidak menyimpan copy harga
Di baris mana message antar-object terjadi?	menu.isTersedia() dan menu.getHarga() di dalam method Pesanan
Mengapa p1 dan p2 bisa punya pemesan berbeda?	Karena masing-masing Pesanan menyimpan reference ke object Mahasiswa yang berbeda

Tahap 5 – Uji Static Secara Sengaja

Pesanan.java

```
public class Pesanan {
    private static int nextNumber = 1;

    private int      nomor;
    private Mahasiswa pemesan;
    private MenuItem menu;
    private int      jumlah;

    public Pesanan(Mahasiswa pemesan, MenuItem menu, int jumlah) {
```

```

        this.nomor    = nextNumber++;
        this.pemesan = pemesan;
        this.menu     = menu;
        this.jumlah   = jumlah;
    }

    public int getNomor() { return nomor; }

    public boolean dapatDiproses() { return jumlah > 0 && menu.isTersedia(); }

    public int hitungTotal() { return menu.getHarga() * jumlah; }

    public void ringkasan() {
        System.out.println("=== Pesanan #" + nomor + " ===");
        System.out.println("Pemesan : " + pemesan.getNama() + " (" + pemesan.getNim() + ")");
        System.out.println("Menu    : " + menu.getNama());
        System.out.println("Jumlah : " + jumlah);
        System.out.println("Total  : Rp " + hitungTotal());
        System.out.println("Status : " + (dapatDiproses() ? "Dapat diproses" : "Ditolak"));
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {

        // Mahasiswa
        Mahasiswa m1 = new Mahasiswa("241001", "Asep");
        Mahasiswa m2 = new Mahasiswa("241002", "Siti");

        // MenuItem
        MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
        MenuItem kopi = new MenuItem("M02", "Kopi Susu", 12000);
        MenuItem mie = new MenuItem("M03", "Mie Ayam", 15000);

        kopi.tandaiHabis();

        // Tiga Pesanan [BARU: tambah p3]
        Pesanan p1 = new Pesanan(m1, nasi, 2);
        Pesanan p2 = new Pesanan(m2, kopi, 1);
        Pesanan p3 = new Pesanan(m1, mie, 3);

        // Uji static counter
        System.out.println("=== Uji Static Counter ===");
        System.out.println("Nomor P1: " + p1.getNomor());
        System.out.println("Nomor P2: " + p2.getNomor());
        System.out.println("Nomor P3: " + p3.getNomor());
        System.out.println();

        System.out.println("=== Uji dapatDiproses ===");
        System.out.println("P1 dapat diproses: " + p1.dapatDiproses());
        System.out.println("P2 dapat diproses: " + p2.dapatDiproses());
        System.out.println("P3 dapat diproses: " + p3.dapatDiproses());
        System.out.println();

        System.out.println("=== Ringkasan Semua Pesanan ===");
        p1.ringkasan();
        System.out.println();
        p2.ringkasan();
        System.out.println();
        p3.ringkasan();
    }
}

```

Output:

```

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Uji Static Counter ===
Nomor P1: 1
Nomor P2: 2
Nomor P3: 3

=== Uji dapatDiproses ===
P1 dapat diproses: true
P2 dapat diproses: false
P3 dapat diproses: true

=== Ringkasan Semua Pesanan ===
=== Pesanan #1 ===
Pemesan : Asep (241001)
Menu    : Nasi Goreng
Jumlah  : 2
Total   : Rp 36000
Status  : Dapat diproses

=== Pesanan #2 ===
Pemesan : Siti (241002)
Menu    : Kopi Susu
Jumlah  : 1
Total   : Rp 12000
Status  : Ditolak

=== Pesanan #3 ===
Pemesan : Asep (241001)
Menu    : Mie Ayam
Jumlah  : 3
Total   : Rp 45000

```

Eksperimen 1 – Private vs Public

Uji kondisi private:

```

7 public class Main {
8     public static void main(String[] args) {
31         MenuItem nasi = new MenuItem(kode: "M01", nama: "Nasi Goreng", harga: 18000);
32         MenuItem kopi = new MenuItem(kode: "M02", nama: "Kopi Susu", harga: 12000);
33         MenuItem mie = new MenuItem(kode: "M03", nama: "Mie Ayam", harga: 15000);
34
35         kopi.tersedia = false;
36     }
}

```

Problems Output Debug Console Terminal Ports Spell Checker 1 Error Comments

```

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>javac id/ac/polban/pbo/kantin/app/Main.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
id\ac\polban\pbo\kantn\app\Main.java:35: error: tersedia has private access in MenuItem
        kopi.tersedia = false;
            ^
1 error

```

Ubah sementara ke public:

```

id > ac > polban > pbo > kantin > app > J Main.java > Main > main(String[])
7 public class Main {
8     public static void main(String[] args) {
31         MenuItem nasi = new MenuItem(kode: "M01", nama: "Nasi Goreng", harga: 18000);
32         MenuItem kopi = new MenuItem(kode: "M02", nama: "Kopi Susu", harga: 12000);
33         MenuItem mie = new MenuItem(kode: "M03", nama: "Mie Ayam", harga: 15000);
34
35         kopi.tersedia = false;
36     }
}

```

Problems Output Debug Console Terminal Ports Spell Checker 0 Comments

```

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>java id.ac.polban.pbo.kantin.app.Main
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Uji Static Counter ===
Nomor P1: 1
Nomor P2: 2
Nomor P3: 3

=== Uji dapatDiproses ===
P1 dapat diproses: true
P2 dapat diproses: false
P3 dapat diproses: true

```

Pertanyaan	Jawaban
Apa yang berubah ketika private menjadi public?	Compiler engiinkan akses dan modifikasi langsung tanpa melalui method
Apakah public membuat program lebih fleksibel? Risiko desain apa yang muncul?	Fleksibel tapi tidak aman karena tidak ada yang menjaga konsistensi nilai, tidak ada validasi, dan perubahan bisa terjadi dari manapun
Mengapa tandaiHabis() lebih bermakna daripada akses langsung ke field?	Karena memberi nama yang jelas pada aksi yang terjadi, dan kalau suatu saat perlu menambahkan logika lain cukup ubah hanya di satu tempat

Eksperimen 2 – Constructor vs Object Setengah Jadi

Coba buat object tanpa argument. Akan terjadi error karena setelah mendefinisikan konstruktor berparameter object Mahasiswa wajib dibuat dengan NIM dan nama.

```

id > ac > polban > pbo > kantin > app > J Main.java > Main > main(String[])
1 package id.ac.polban.pbo.kantin.app;
2
3 import id.ac.polban.pbo.kantin.model.Mahasiswa;
4 import id.ac.polban.pbo.kantin.model.MenuItem;
5 import id.ac.polban.pbo.kantin.model.Pesanan;
6
7 public class Main {
8     public static void main(String[] args) {
9         // Mahasiswa
10        Mahasiswa m1 = new Mahasiswa(nim: "241001", nama: "Asep");
11        Mahasiswa m2 = new Mahasiswa(nim: "241002", nama: "Siti");
12        Mahasiswa mCoba = new Mahasiswa();
13        // MenuItem
14        MenuItem nasi = new MenuItem(kode: "M01", nama: "Nasi Goreng", harga: 18000);
15        MenuItem kopi = new MenuItem(kode: "M02", nama: "Kopi Susu", harga: 12000);
16    }
17 }
18
Problems Output Debug Console Terminal Ports Spell Checker
D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>javac id/ac/polban/pbo/kantin/app/Main.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
id/ac/polban/pbo/kantin/app/Main.java:12: error: constructor Mahasiswa in class Mahasiswa cannot be applied to given types;
    Mahasiswa mCoba = new Mahasiswa();
                        ^
    required: String,String
    found:    no arguments
    reason: actual and formal argument lists differ in length
1 error

```

Tambah sementara constructor kosong dan membuat object tanpa NIM/nama

```

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>javac id/ac/polban/pbo/kantin/app/Main.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>java id.ac.polban.pbo.kantin.app.Main
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
NIM : null
Nama : null

```

Object bisa dibuat ketika constructor kosong diizinkan

Eksperimen 3 – Static vs Instance

Membuktikan perbedaan perilaku nextNumber saat static vs tidak static

```

=== Uji Static Counter ===
Nomor P1: 1
Nomor P2: 2
Nomor P3: 3

=== Uji Static Counter ===
Nomor P1: 1
Nomor P2: 1
Nomor P3: 1

```

	static	non-static
nextNumber disimpan dimana?	Satu tepat milik class	Di setiap object, masing masing punya Salinan
Saat p1 dibuat	nextNumber = 1 lalu jadi 2	nextNumber milik p1 = 1 lalu jadi 2
Saat p2 dibuat	nextNumber yang sama sekarang = 2 lalu jadi 3	nextNumber baru milik p2, mulai dari 1 lagi
Hasil nomor	1,2,3	1,1,1

Eksperimen 4 – Relationship: Reference vs Duplikasi Data

Dua cara menyimpan informasi pada Pesanan

```
// Desain A
private Mahasiswa pemesan;
private MenuItem menu;

// Desain B
private String nimPemesan;
private String namaPemesan;
private String kodeMenu;
private String namaMenu;
private int hargaMenu;
```

Pertanyaan	Jawaban
Desain mana yang menunjukkan relationship secara langsung?	Desain A field bertipe Mahasiswa dan MenuItem secara eksplisit menyatakan bahwa Pesanan <i>mengenal</i> kedua object itu
Jika nama mahasiswa berubah, desain mana yang menyimpan copy lama?	Desain B karena data disalin sebagai String, perubahan di object Mahasiswa asli tidak ikut terefleksikan di Pesanan
Apakah reference selalu lebih benar?	Tidak selalu, jika perlu merekam snapshot data <i>pada saat pesanan dibuat</i> (misalnya untuk laporan historis yang tidak boleh berubah), menyimpan copy data adalah pilihan yang disengaja dan valid

Eksperimen 5 – Package dan Compiler

Hapus satu import lalu compile

```
id > ac > polban > pbo > kantin > app > Main.java > ...
1 package id.ac.polban.pbo.kantin.app;
2
3 import id.ac.polban.pbo.kantin.model.Mahasiswa;
4 //import id.ac.polban.pbo.kantin.model.MenuItem;
5 import id.ac.polban.pbo.kantin.model.Pesanan;
6
7 public class Main {
8     public static void main(String[] args) {
9         // Mahasiswa
10         Mahasiswa m1 = new Mahasiswa(nim: "241001", nama: "Asep");
11         Mahasiswa m2 = new Mahasiswa(nim: "241002", nama: "Siti");
12         // MenuItem
13         MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
14         MenuItem kopi = new MenuItem("M02", "Kopi Susu", 12000);
15         MenuItem mie = new MenuItem("M03", "Mie Ayam", 15000);
16
17         kopi.tandaiHabis();
18
19         // Pesanan
20         Pesanan p1 = new Pesanan(m1, nasi, jumlah: 2);
21         Pesanan p2 = new Pesanan(m2, kopi, jumlah: 1);
22     }
23 }
```

Problems 1 Output Debug Console Terminal Ports Spell Checker 13 JavaSE-25 LTS

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>javac id/ac/polban/pbo/kantin/app/Main.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
id\ac\polban\pbo\kant\app\Main.java:13: error: cannot find symbol
 MenuItem nasi = new MenuItem("M01", "Nasi Goreng", 18000);
 ^
symbol: class MenuItem

Compiler tidak tahu class MenuItem ada di mana. Tanpa import, ain tidak tau harus mencari namespace mana

Independent Challenge A

Has-a → object lain disimpan sebagai field.

(Contoh: Pesanan menyimpan Mahasiswa dan MenuItem secara permanen)

Uses-a → object lain hanya diterima sebagai parameter method

(Contoh: Kasir hanya memegang Pesanan sesaat ketika proses() dipanggil. Setelah method selesai, Kasir tidak menyimpan apapun tentang Pesanan itu)

Membuat file baru Kasir.java
<pre>package id.ac.polban.pbo.kantin.app; import id.ac.polban.pbo.kantin.model.Pesanan; public class Kasir { private String namaKasir; public Kasir(String namaKasir) { this.namaKasir = namaKasir; } public String getNamaKasir() { return namaKasir; } // uses-a: Pesanan HANYA diterima sebagai parameter // Kasir tidak menyimpan Pesanan sebagai field // Setelah method ini selesai, Kasir tidak "ingat" pesanan apapun public void proses(Pesanan pesanan) { System.out.println("Kasir [" + namaKasir + "] " + "memproses Pesanan #" + pesanan.getNomor()); if (pesanan.dapatDiproses()) { System.out.println(" Hasil : DITERIMA"); System.out.println(" Total : Rp " + pesanan.hitungTotal()); } else { System.out.println(" Hasil : DITOLAK"); System.out.println(" Alasan : Menu habis atau jumlah tidak valid"); } } // uses-a Pesanan masuk sebagai parameter, bukan field public void cetakStruk(Pesanan pesanan) { if (!pesanan.dapatDiproses()) { System.out.println(" [Struk tidak dicetak – pesanan ditolak]"); return; } System.out.println(" ===== STRUK KANTIN POLBAN ====="); System.out.println(" Kasir : " + namaKasir); System.out.println(" No. : " + pesanan.getNomor()); System.out.println(" Pemesan: " + pesanan.getPemesan().getNama() + " (" + pesanan.getPemesan().getNim() + ")"); System.out.println(" Menu : " + pesanan.getMenu().getNama()); System.out.println(" Total : Rp " + pesanan.hitungTotal()); System.out.println(" ====="); } } // Kasir = CLASS kasir1 = OBJECT/INSTANCE // Kasir punya state sendiri (namaKasir)</pre>
Tambahan di Main.java
<pre>System.out.println(); System.out.println("=== Challenge A: uses-a dengan Kasir ==="); // Kasir = CLASS kasir1 = OBJECT/INSTANCE // Kasir punya state sendiri (namaKasir)</pre>

```
// tapi TIDAK menyimpan Pesanan sebagai field
Kasir kasir1 = new Kasir("Budi");
Kasir kasir2 = new Kasir("Ani");

System.out.println();
System.out.println("-- Kasir " + kasir1.getNamaKasir() + " memproses --");
kasir1.proses(p1);
System.out.println();
kasir1.proses(p2);
System.out.println();
kasir1.proses(p3);

System.out.println();
System.out.println("-- Kasir " + kasir2.getNamaKasir() + " mencetak struk --");
kasir2.cetakStruk(p1);
System.out.println();
kasir2.cetakStruk(p2);
```

```
=== Challenge A: uses-a dengan Kasir ===

-- Kasir Budi memproses --
Kasir [Budi] memproses Pesanan #1
  Hasil : DITERIMA
  Total : Rp 36000

Kasir [Budi] memproses Pesanan #2
  Hasil : DITOLAK
  Alasan : Menu habis atau jumlah tidak valid

Kasir [Budi] memproses Pesanan #3
  Hasil : DITOLAK
  Alasan : Menu habis atau jumlah tidak valid

-- Kasir Ani mencetak struk --
===== STRUK KANTIN POLBAN =====
Kasir : Ani
No. : 1
Pemesan: Asep (241001)
Menu : Nasi Goreng
Total : Rp 36000
=====

[Struk tidak dicetak – pesanan ditolak]

D:\POLBAN-2025\smt3\PBO\Praktek\Praktikum3>
```

Independent Challenge B

Jika nanti terdapat MenuMakanan dan MenuMinuman, apakah keduanya is-a MenuItem?

Iya, keduanya merupakan bagian atau versi yang lebih spesifik dari MenuItem dan mewarisi semua karakteristik MenuItem (punya kode, nama, harga, status tersedia/habis, bisa ditandai habis)